Semantic Search for Hindi Documents using Sentence Embeddings

A Comparative Study of TF-IDF and Semantic Retrieval

Abhinav Marlingaplar 2023BCD0013
Saksham Saklani 2023BCD0049

Indian Institute of Information Technology Kottayam · B.Tech in Computer Science and Engineering (Specialization: Artificial Intelligence and Data Science) · October 2026

Abstract. A search system should find documents that match what a user means, not only the words they typed. We compare *keyword retrieval* (TF-IDF, BM25), which scores documents by shared words, with *semantic retrieval*, which compares sentence embeddings (lists of numbers that capture meaning), on a Hindi duplicate-question collection of 5,046 questions, 50 queries and 70 relevance judgments. Six methods are scored with nDCG@10, MRR@10 and Recall@10, with bootstrap confidence intervals and paired significance tests. Multilingual E5 reaches nDCG@10 $= 0.893$, against 0.692 for the best TF-IDF and 0.658 for BM25, and both embedding models beat BM25 by margins that survive a multiple-comparison correction. Equal-weight fusion of BM25 and E5 *lowers* quality relative to E5 alone. The default scikit-learn tokenizer cuts Devanagari words into fragments, although fixing it gave no statistically clear gain. Embeddings win on paraphrases and spelling variants; some failures come from translation inconsistencies in the data.

1 Introduction

Search engines, FAQ assistants and duplicate-question detectors share one task: given a *query*, rank a collection of documents so that the most relevant come first. Classical systems match *words*, which fails when the same intent is written differently. *Semantic retrieval* instead uses a neural sentence encoder to turn each sentence into a vector (an *embedding*) so that sentences with similar meaning receive similar vectors; documents are ranked by their closeness to the query, even when no word is shared (Figure 1).

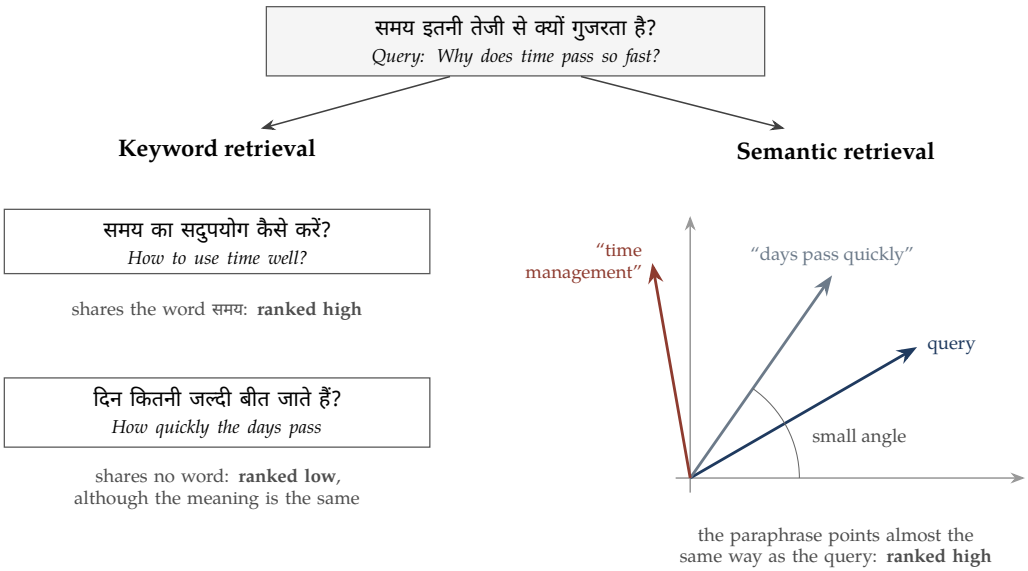


Figure 1. Keyword versus semantic retrieval (illustrative example, not from the dataset). Embeddings are drawn as arrows in a two-dimensional sketch of the embedding space.

Hindi is a good test case. It is written in Devanagari, where vowels are often attached to consonants as signs (*matras*) and a joining mark (*virama*) fuses consonants; words inflect heavily; and retrieval

resources are limited compared with English [1, 2]. Two recent papers motivated this work: Hindi-BEIR [1] reports multilingual E5 above BM25 on Hindi retrieval (mean nDCG@10 44.37 vs. 35.65), and IndicIRSuite [2] reports a neural model above BM25 on MIRACL Hindi (0.483 vs. 0.458). We adopt their metric (nDCG@10) and baselines on a smaller task, but do not train their proposed models (NLLB-E5, Indic-ColBERT).

Research question. *Can sentence-embedding-based semantic retrieval provide more meaningful and effective Hindi document retrieval than traditional TF-IDF-based retrieval?*

Project history. At the midterm we built TF-IDF and MiniLM pipelines and compared them by eye on 8 of the 50 queries; no metric had been computed. For the end-term we audited the TF-IDF baseline, added BM25, a stronger encoder (multilingual E5) and a hybrid method, scored all 50 queries with standard metrics, tested whether differences are statistically reliable, and analysed the errors. We also revisited two midterm observations (Section 6).

2 Dataset

We use `Bharat_NanoQuoraRetrieval_hi` [12], a Hindi Quora-style duplicate-question collection from the Hugging Face Hub. A query is a question; a document is *relevant* if it asks the same thing. It has **5,046 corpus questions**, **50 queries** and **70 binary relevance judgments** (1.4 per query; 40 queries have exactly one). Most questions are under 100 characters. Because relevant pairs are duplicates, they often differ in wording, which is where keyword and semantic retrieval should behave differently.

Text is only normalised (Unicode NFC and whitespace); we apply no stemming or stop-word removal, so the two pipelines stay comparable. Two checks guard the evaluation. A query that also appears in the corpus would trivially retrieve itself, so such matches would be excluded (none occurred). Three queries have a textually identical relevant document; they are easy for every method, and removing them leaves the ordering of methods unchanged (E5 0.886, MiniLM 0.835, hybrid 0.761, TF-IDF fixed 0.675, BM25 0.643, TF-IDF default 0.633).

3 Methodology

All methods sit behind one scoring function: given a query and a method it scores each of the 5,046 documents, ranks them and keeps the top 100, so evaluation and demonstrations use identical code (Figure 2). The system is a single Python notebook (Google Colab, GPU) using scikit-learn, `rank_bm25`, sentence-transformers and `ranx`; document embeddings are cached and the only random step, the bootstrap, uses seed 42.

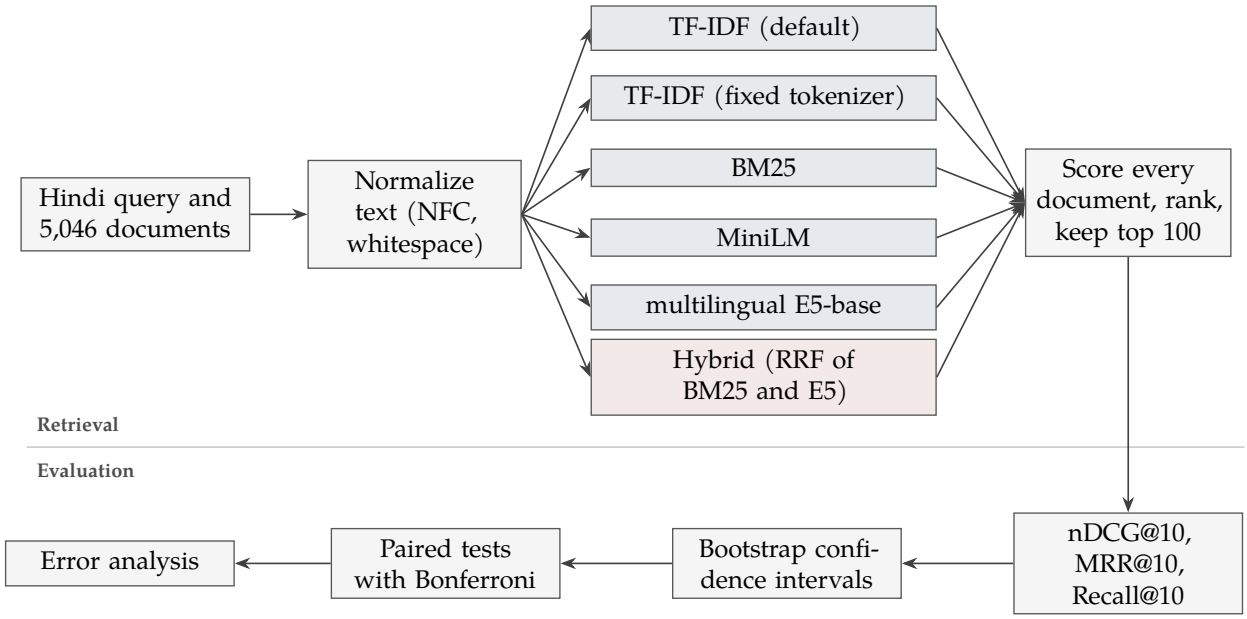


Figure 2. System pipeline: retrieval (top) produces a ranked list for each method; evaluation (bottom) measures and compares them.

3.1 Keyword methods

TF-IDF turns each document into a vector with one weight per word, large when the word is frequent in the document but rare in the corpus; documents are ranked by cosine similarity with the query. **BM25** [7] refines this with term-frequency saturation and length normalisation. We run TF-IDF with scikit-learn's default settings (our midterm baseline) and with a fixed tokenizer, and BM25 from `rank_bm25` (default $k_1 = 1.5$, $b = 0.75$, fixed tokenizer). Both only match *words*.

Tokenization of Devanagari. scikit-learn's default token pattern treats only letters as word characters. Vowel signs and the virama are combining marks, so the pattern cuts a Hindi word at each mark and discards one-letter fragments (Figure 3). Words containing a vowel sign or virama are 90% of the corpus, and the default tokenizer keeps none of them intact (0.0%, against 96.4% for the other words); the vocabulary grows from 1,556 to 6,172 terms after the fix.

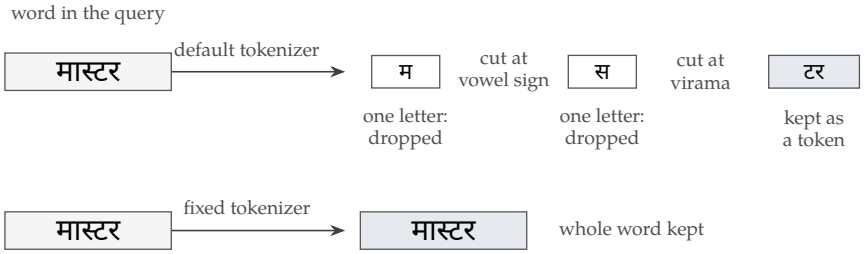


Figure 3. How the two tokenizers treat a Hindi word.

3.2 Semantic methods

A *sentence encoder* is a neural network, pre-trained on large amounts of text, that maps a sentence to a vector of a few hundred numbers so that sentences with the same meaning lie close together. A *multilingual* encoder does this for many languages, so Hindi can be encoded without Hindi-specific training on our side. We use two pre-trained encoders with no fine-tuning: **MiniLM** (`paraphrase-multilingual-MiniLM-L12-v2`, 384 dimensions) [5, 6] and **multilingual E5-base** (`intfloat/multilingual-e5-base`, 768 dimensions) [4]. E5 expects the prefixes "query: " and "passage: " on its input, which we add. All documents are encoded once; each query is encoded at search time, vectors are length-normalised, and documents are ranked by cosine similarity.

3.3 Hybrid: reciprocal rank fusion

Reciprocal Rank Fusion (RRF) [8] merges rankings: a document earns $1/(60 + \text{rank})$ points from each method and the points are added, so a document ranked high by both wins.

$$\text{RRF}(d) = \sum_{m \in \{\text{BM25, E5}\}} \frac{1}{60 + \text{rank}_m(d)} \tag{1}$$

Both methods have equal weight and nothing was tuned. E5 was chosen as the partner because it beat MiniLM on our queries (a choice made on the evaluation data, which can only favour the hybrid).

4 Evaluation Protocol

Metrics. For each query we score the ranked list against its relevant set R at cutoff $k = 10$ (all between 0 and 1, higher is better): *Recall@10* is the share of R found in the top 10; *MRR@10* is $1/r$, where r is the rank of the first relevant document (0 if none); and *nDCG@10* rewards placing relevant documents high, nDCG@k = DCG@k/IDCG@k with DCG@$k = \sum_{i=1}^{k} \text{rel}_i / \log_2(i+1)$ [9]. nDCG@10 is our main metric, as in BEIR and Hindi-BEIR [3, 1]. For a single relevant document at rank 1, 2 or 3, nDCG@10 is 1.00, 0.63 or 0.50. Precision@K is omitted: with about 1.4 relevant documents per query it cannot exceed about 0.14 and adds no information.

Validation. We implemented the metrics ourselves and checked them: a perfect ranking scores 1, a ranking with no relevant document scores 0, and our values match the `ranx` library [11] to six decimals.

Uncertainty. With only 50 queries a method can look better by luck, so each nDCG@10 carries a 95% *bootstrap* confidence interval [10]: resample the 50 queries with replacement, recompute the mean, repeat 2,000 times, and take the 2.5th and 97.5th percentiles. To compare two methods we bootstrap the paired difference per query; if its 95% interval excludes 0 the difference is *significant*. Because we make eight comparisons, we also report a *Bonferroni* correction (99.375% intervals); a result that survives it is very likely real.

Error analysis. We compare BM25 with E5 query by query. A method *wins* a query only if its nDCG@10 is higher by at least 0.10 (our own threshold); otherwise the query is comparable (both solve it, or both struggle).

5 Results

Table 1. Retrieval quality on 50 queries (95% bootstrap CI for nDCG@10; best value in bold).

Method	nDCG@10 [95% CI]	MRR@10	Recall@10
TF-IDF (default, midterm)	0.649 [0.547, 0.751]	0.619	0.816
TF-IDF (fixed tokenizer)	0.692 [0.585, 0.790]	0.676	0.816
BM25	0.658 [0.548, 0.763]	0.649	0.773
MiniLM	0.845 [0.765, 0.913]	0.826	0.953
Multilingual E5-base	**0.893** [0.826, 0.949]	**0.877**	**0.977**
Hybrid (RRF of BM25 and E5)	0.775 [0.678, 0.859]	0.747	0.903

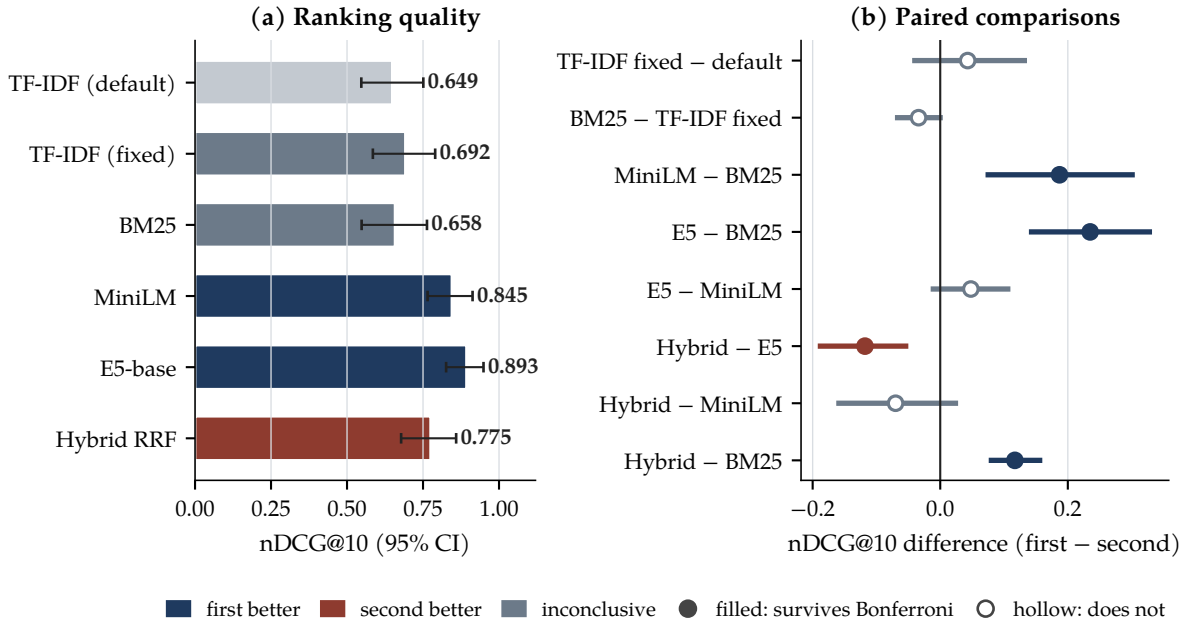


Figure 4. (a) nDCG@10 with 95% confidence intervals. (b) Paired differences between methods with 95% intervals; an interval that does not cross zero is a significant difference.

Multilingual E5 is best on all three metrics, followed by MiniLM; both are clearly ahead of every keyword method. Sixteen of the 50 queries are solved perfectly by every method, so the differences come from the harder queries. Table 2 gives the paired comparisons.

Table 2. Paired bootstrap comparisons of nDCG@10. "Survives" = significant after the Bonferroni correction.

First − second method	Difference	95% interval	p	Survives
TF-IDF fixed − TF-IDF default	$+0.043$	$[-0.044, +0.136]$	0.357	no
BM25 − TF-IDF fixed	-0.034	$[-0.071, +0.004]$	0.081	no
MiniLM − BM25	$+0.187$	$[+0.071, +0.305]$	0.001	**yes**
E5 − BM25	$+0.235$	$[+0.139, +0.332]$	$<$0.001	**yes**
E5 − MiniLM	$+0.048$	$[-0.015, +0.110]$	0.140	no
Hybrid − E5	-0.118	$[-0.192, -0.050]$	$<$0.001	**yes**
Hybrid − MiniLM	-0.070	$[-0.163, +0.028]$	0.152	no
Hybrid − BM25	$+0.117$	$[+0.076, +0.160]$	$<$0.001	**yes**

Significant: E5 beats BM25 by 0.235 and MiniLM beats it by 0.187, so the advantage of embeddings is not specific to one encoder. The hybrid is significantly *worse* than E5 alone (-0.118) though better than BM25 alone $(+0.117)$: combining helps a weak method but not a strong one. **Inconclusive:** the tokenizer fix, BM25 versus TF-IDF, E5 versus MiniLM and the hybrid versus MiniLM all have intervals that include zero. These are not proofs of equality: with 50 queries our intervals extend about 0.04 to 0.10 on each side, so smaller differences cannot be resolved. In particular E5 is ahead of MiniLM numerically, but we cannot say it is better.

6 Analysis

Tokenizer. The fragmenting of Devanagari words is severe (Section 3.1), yet its effect on ranking is small and inconclusive: TF-IDF rises from 0.649 to 0.692 but the interval includes zero. A possible explanation, which we did not test, is that the surviving fragments still carry signal on short questions and that 50 queries give little power to detect a small gain.

Where each method wins. Comparing BM25 with E5, embeddings win on 21 queries and keyword search on 2; 24 queries are solved by both and 3 are hard for both. Table 3 shows representative cases. Embeddings win when the same meaning is written with different words or spellings. Keyword search wins rarely, on generic queries where E5 ranks questions about specific topics above the generic duplicate. The hard queries are partly a data problem: two of three appear to be translation inconsistencies of one English question (the last row seems to render "demonetization" as "demonization"), and the third is too vague to have one answer.

Table 3. Example queries; ranks give the position of the first relevant document. English glosses are ours.

Case	Query	Labelled relevant document	BM25	E5	Hybrid
Embedding wins: spelling	मैं अपने कैरियर के हित का निर्धारण कैसे करूं? (How do I determine my career interests?)	मैं अपने करियर के हितों का पता कैसे लगा सकता हूँ? (How can I find out my career interests?)	68	1	7
Embedding wins: paraphrase	क्या रमा मांस खाता है? (Does Rama eat meat?)	क्या भगवान राम मांस खाते थे? (Did Lord Ram eat meat?)	15	1	4
Keyword wins: generic	इसका इलाज कैसे किया जा सकता है? (How can this be treated?)	यदि नहीं, तो इसका इलाज कैसे किया जा सकता है? (If not, how can this be treated?)	1	2	1
Hard: label noise	दानवीकरण के क्या-क्या पक्ष और विपक्ष हैं? (Pros and cons of "दानवीकरण")	नोटबंदी के क्या-क्या फायदे और नुकसान हैं? (Advantages and disadvantages of demonetisation)	5	9	4

Why fusion hurt. The hybrid is worse than E5 on 13 queries, similar on 34 and better on only 3. In nine queries E5 ranks a relevant document first but the hybrid does not; in the eight most extreme, BM25 ranks it between 7th and 68th. RRF gives both methods an equal vote, so a poor BM25 rank can outweigh an excellent E5 rank: a relevant document ranked 68th by BM25 and 1st by E5 scores $\frac{1}{128} + \frac{1}{61} = 0.0242$,

below a non-relevant document ranked 1st by BM25 and 5th by E5 ($\frac{1}{61} + \frac{1}{65} = 0.0318$). This is our hypothesis, supported by these cases; we did not test weighted fusion, so it applies to equal-weight RRF only.

Midterm observations revisited. At the midterm TF-IDF seemed better on a distinctive term (Q1, स्क्रम मास्टर क्या है?) and MiniLM better on a paraphrase (Q8). With all methods on the same queries, fixed TF-IDF, BM25 and E5 all rank the Scrum Master question first and only MiniLM ranks it third, so the Q1 result was specific to MiniLM. For Q8, BM25 and E5 return the same top question as MiniLM, while TF-IDF returns a weaker one, so the contrast came from TF-IDF's weighting, not keyword search in general. Both observations were real, but they came from one query, one baseline and one encoder, which is why our conclusions rest on the 50-query evaluation.

7 Discussion and Conclusion

Answer. For short Hindi questions, sentence-embedding retrieval is clearly more effective than keyword retrieval: multilingual E5 reaches nDCG@10 $= 0.893$ against 0.692 for the best TF-IDF and 0.658 for BM25, mainly because it matches meaning when the wording differs. The direction agrees with the reference papers, but our gap is larger (+23.5 points against +8.7 in Hindi-BEIR and +2.5 in IndicIR-Suite). A possible reason, untested, is that duplicate-question pairs are paraphrases by construction, texts are short and the corpus is small; absolute numbers from different datasets are not comparable. Table 4 lists what the evidence supports.

Table 4. Findings and the strength of the evidence behind each.

Finding	Evidence	Strength
Embeddings beat keyword search	+0.19 to +0.24 nDCG@10; embeddings win on 21 queries, keyword search on 2	Supported
The gain comes from paraphrases and wording differences	Relevant documents at BM25 rank 68 but E5 rank 1	Supported (examples)
Equal-weight fusion hurts E5	-0.118; worse on 13 queries, better on 3	Supported (this scheme)
The default tokenizer breaks Devanagari words	0% of such words kept; 90% of the corpus	Supported
That bug lowers ranking quality	+0.043; interval includes 0	Not shown
E5-base is better than MiniLM	+0.048; interval includes 0	Not shown
BM25 is better than TF-IDF	-0.034 (opposite sign); interval includes 0	Not shown

Recommendation. For short Hindi question retrieval, use a multilingual embedding model on its own and skip the simple hybrid. MiniLM is smaller and, on our data, not shown to be worse than E5-base, so it is a sensible choice when speed matters.

Limitations. Only 50 queries and one dataset of short questions, so we cannot claim the results hold for long documents, other domains or other Indian languages, and we tried only one way of combining methods. The data are machine-translated and contain some inconsistent pairs. Labels are incomplete (about 1.4 per query), so a good unlabelled match counts as an error and lowers every score. The corpus is small, so absolute scores are higher than on a full-size search. The hybrid's partner was chosen on the evaluation queries, and no model was fine-tuned. Of the Hindi-specific challenges named by Hindi-BEIR (script, inflection, ambiguity) we examined only script handling.

Future work. Weighted fusion tuned on separate queries, a reranking stage, evaluation on further Hindi-BEIR datasets with longer documents, reproducing NLLB-E5, and searching Hindi documents with English queries.

Conclusion. The project moved from a qualitative midterm impression to a statistically checked comparison. Semantic retrieval with multilingual sentence embeddings clearly outperforms TF-IDF and BM25 on this dataset; a simple equal-weight hybrid does not help; the default TF-IDF tokenizer fragments Devanagari words without a measurable effect on ranking here; and several smaller differences cannot be resolved with 50 queries. We report both what the data support and what they do not.

References

[1] A. Acharya, R. Murthy, V. Kumar, and J. Sen, "Benchmarking and building zero-shot Hindi retrieval model with Hindi-BEIR and NLLB-E5," in *Proc. NAACL-HLT* (*Long Papers*), 2025, pp. 4328–4348.

[2] S. Haq, A. Sharma, O. Khattab, N. Chhaya, and P. Bhattacharyya, "IndicIRSuite: Multilingual dataset and neural information models for Indian languages," in *Proc. ACL* (*Short Papers*), 2024, pp. 501–509.

[3] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *NeurIPS Datasets and Benchmarks*, 2021.

[4] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 text embeddings: A technical report," arXiv:2402.05672, 2024.

[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019.

[6] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," in *Proc. EMNLP*, 2020.

[7] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.

[8] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. SIGIR*, 2009, pp. 758–759.

[9] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM Trans. Inf. Syst.*, vol. 20, no. 4, pp. 422–446, 2002.

[10] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[11] E. Bassani, "ranx: A blazing-fast Python library for ranking evaluation and comparison," in *Proc. ECIR*, 2022, pp. 259–264.

[12] "Bharat_NanoQuoraRetrieval_hi," Hugging Face dataset `carlfeynman/Bharat_NanoQuoraRetrieval_hi`.